Reinforcement Learning for Quantum Control

Morten Hjorth-Jensen

March 17, 2026

Motivation

- Quantum sensors permit fine-grained control over probe evolution and measurement settings.
- The central design problem is sequential: which control should be applied *next*, given what has already been learned?
- For realistic platforms, the estimation stack contains many non-smooth components: sampling of measurement outcomes, particle-filter resampling, and optional weak-measurement backaction.
- The article proposes a model-aware RL framework that treats experimental design as a trainable control problem while retaining a faithful physical model.

Talk focus: the technical machinery in Appendices A–I, with figures intentionally omitted so they can be inserted later.

Main thesis of the paper

Core idea

Use a differentiable simulation of the experiment, Bayesian inference by particle filtering, and policy optimization to learn adaptive or non-adaptive control laws for quantum metrology.

- **Bayesian mode:** optimize the final estimation error after a complete measurement sequence.
- **Frequentist mode:** optimize the Cramer-Rao bound through the Fisher information matrix.
- **Model-aware:** gradients are propagated through a known sensor model rather than inferred only from trial-and-error interaction.
- **Applications in the paper:** NV-center DC magnetometry and the agnostic Dolinar receiver.

The measurement loop as a control architecture

1. From the current posterior summary, the agent outputs the next control x_{t+1}.
2. The probe evolves and is measured, producing a stochastic outcome y_{t+1}.
3. The Bayesian posterior is updated and fed back to the agent.

Adaptive policies are written as

$$x_{t+1} = F_\lambda\big(P(\theta \mid x_t, y_t);\ y_t;\ R_t;\ t\big),$$

while non-adaptive strategies suppress dependence on the posterior and outcomes:

$$x_{t+1} = F_\lambda(R_t, t).$$

- λ: trainable parameters of the policy (typically a neural network).
- R_t: resources consumed up to step t (time, photons, signal amplitude, ...).

Appendix A: probe, controls, and measurement model

A generic experiment is cast as

$$\rho \longmapsto \rho_{x,\theta} = \mathcal{E}_{x,\theta}(\rho),$$

where ρ is a reference probe state, x denotes tunable controls, and $\theta \in \Theta$ are the unknown parameters.

- Controls can be continuous or discrete, and may represent durations, detunings, phases, or measurement settings.
- Parameters of interest can coexist with *nuisance parameters*; estimating the latter can still improve the former.
- A measurement is modeled by a POVM $\mathcal{M}^x = \{M_y^x\}$ with likelihood

$$p(y \mid \theta, x) = \mathrm{tr}\left(M_y^x \rho_{x,\theta}\right).$$

Appendix A: projective versus weak measurements

- For **projective measurements**, the probe is reinitialized after each shot; the next likelihood depends only on x_{t+1} and θ.
- For **weak measurements**, information remains in the probe state, so the likelihood depends on the whole trajectory:

$$p(y_{t+1} \mid x_{t+1}, y_t, \theta) = \mathrm{tr}\left(M_{y_{t+1}}^{x_{t+1}} \rho_{x_t, y_t, \theta}\right).$$

- This turns the estimator into a partially observed control problem in which the posterior and probe state both carry memory.

Control lesson: the state fed to the agent is not the raw experimental history, but a compressed belief-state summary derived from Bayesian inference.

Appendix B: particle-filter representation of the posterior

The posterior is approximated by a weighted ensemble

$$P(\theta) \approx \sum_{j=1}^{N} w_j^t \, \delta(\theta - \theta_j),$$

with particles $\{\theta_j\}_{j=1}^N$ and weights $\{w_j^t\}_{j=1}^N$.

- Initialize particles from the prior $\pi(\theta)$ and set $w_j^0 = 1/N$.
- Apply Bayes updates after each observation:

$$w_j^{t+1} = \frac{p(y_{t+1} \mid x_{t+1}, \theta_j) w_j^t}{\sum_{i=1}^N p(y_{t+1} \mid x_{t+1}, \theta_i) w_i^t}.$$

- The posterior mean and covariance become

$$\hat{\theta}_t \approx \sum_j w_j^t \theta_j, \qquad \Sigma_t \approx \sum_j w_j^t (\theta_j - \hat{\theta}_t)(\theta_j - \hat{\theta}_t)^\top.$$

These moments form the default summary presented to the policy.

Appendix B.3: resampling, soft resampling, and proposal moves

When weights collapse onto a few particles, the filter is refreshed by resampling.

Soft resampling

Instead of sampling directly from w_j, define

$$q_j = \alpha w_j + (1-\alpha)\frac{1}{N},$$

and draw new particles $\theta'_j = \theta_{\phi(j)}$ from q_j.

Corrected weights preserve the represented posterior:

$$w'_j \propto \frac{w_{\phi(j)}}{q_{\phi(j)}}.$$

Further steps in Appendix B:

- **Gaussian perturbation:** move particles toward $\hat{\theta}_t$ and break degeneracy.
- **New-particle proposal:** inject particles from $\mathcal{N}(\hat{\theta}_t, \Sigma_t)$ to raise local resolution.

Appendix B.4: state particle filter for weak measurements

For weak measurements, each parameter particle carries its own hypothetical probe state:

$$\rho_{\theta_j,\tau_t} = \mathcal{M}_{y_t}^{x_t} \circ \mathcal{E}_{\theta_j,x_t} \circ \cdots \circ \mathcal{M}_{y_1}^{x_1} \circ \mathcal{E}_{\theta_j,x_1}(\rho).$$

The normalized measurement backaction map is

$$\mathcal{M}_y^x(\rho) = \frac{M_y^x \rho M_y^{x\dagger}}{\mathrm{tr}[M_y^x \rho M_y^{x\dagger}]}.$$

- The state particle filter stores $(\theta_j, w_j, \rho_{\theta_j,\tau_t})$.
- It enables trajectory-dependent likelihood evaluation for probes that are not reinitialized between measurements.
- This is the natural extension from Bayesian filtering over parameters to Bayesian filtering over *parameter-conditioned states*.

Why differentiation is hard

The training objective depends on operations that are stochastic or discrete:

- measurement outcomes $y_t \sim p(y_t \mid x_t, \theta)$,
- categorical particle resampling,
- stochastic perturbation and proposal draws,
- optional discrete decision outputs in classification tasks.

Automatic differentiation treats sampled values as constants in the backward pass, so naively

$$\frac{dy_t}{d\lambda} = 0,$$

and the gradient would fail to capture how the policy changes the distribution of future trajectories. **Appendices C and D** supply the fix: combine score-function terms with differentiable surrogates wherever possible.

Appendix C: differentiable particle filtering

Three ingredients make the particle filter trainable.

1. **Soft resampling:** lets some gradient information survive resampling through the weights.
2. **Reparametrization trick:** write perturbations and proposals as differentiable functions of Gaussian noise,

$$\delta'(y_t, \lambda) = \Sigma(y_t, \lambda)u + \mu(y_t, \lambda), \qquad \theta''_j(y_t, \lambda) = \Sigma_t(y_t, \lambda)u_j + \hat{\theta}_t(y_t, \lambda).$$

3. **Scibior-Wood correction:** replace the new weights by the surrogate

$$\tilde{w}'_j \leftarrow w'_j \frac{q_{\phi(j)}}{\mathrm{sg}[q_{\phi(j)}]},$$

   which leaves the forward pass unchanged but injects missing backward terms.

For the MSE loss, the paper shows that this correction reproduces the needed score-function terms.

Appendix D.1–D.3: Bayesian loss and policy-gradient correction

For continuous estimation tasks, the single-run loss is an MSE-type quantity,

$$\ell(\hat{\theta}, \theta) = \mathrm{tr}\Big[G(\hat{\theta} - \theta)(\hat{\theta} - \theta)^\top\Big],$$

while discrete tasks use a 0-1 loss. Monte Carlo over a batch of B simulations gives

$$L(\lambda) = \frac{1}{B}\sum_{k=1}^{B} \ell(\hat{\theta}_k, \theta_k).$$

Because outcomes are sampled from a policy-dependent distribution, the correct training loss becomes

$$\tilde{\ell}(\omega_k, \lambda) = \ell(\omega_k, \lambda) + \mathrm{sg}[\ell(\omega_k, \lambda)] \log p(y_{k,0:M-1} \mid \theta_k, \lambda),$$

so that the gradient includes both pathwise terms and score-function terms.

Appendix D.4–D.6: cumulative and logarithmic losses

A final-time loss alone may not yield good performance at intermediate resource budgets.

Cumulative loss

$$L_{\mathrm{cum}}(\lambda) = \frac{1}{MB} \sum_{t=0}^{M-1} \sum_{k=1}^{B} \ell(\hat{\theta}_{k,t}, \theta_k).$$

This pressures the policy to use resources sensibly *throughout* the experiment.

Logarithmic loss

Taking a logarithm of the batch-averaged loss stabilizes optimization when the precision spans several orders of magnitude.

- A baseline can be subtracted inside the stop-gradient term to reduce gradient variance.
- The paper uses Adam rather than plain SGD for the actual parameter updates.

Appendix E: simulation practice, hyperparameters, and scaling

Practical tuning strategy

1. Choose the number of particles N large enough that posterior discretization is not the bottleneck.
2. Use the remaining memory budget to set the batch size B.
3. Tune the initial learning rate with the chosen loss (cumulative or logarithmic).

Complexity trends reported in Appendix E.3

$$\text{Memory} \sim \mathcal{O}(dN + N + n_\ell n_u), \qquad \text{Time}_{\text{seq}} \sim \mathcal{O}(MN + Mn_\ell n_u^2),$$

for parameter dimension d, measurements M, and a neural network with n_ℓ layers and n_u units per layer.

- Training benefits strongly from GPU parallelism over the batch.
- Deployment can be far cheaper, especially for non-adaptive strategies or offline Bayesian updates.

Applications highlighted in the main text

NV-center DC magnetometry

- Control variable: Ramsey free-evolution time τ.
- Likelihood: binary outcomes with dephasing-dependent contrast.
- Learned strategies beat common heuristics and prior model-free RL baselines in the reported settings.

Agnostic Dolinar receiver

- Task: discriminate coherent states while also treating the amplitude as an unknown nuisance parameter.
- Control variable: beam-splitter reflectivity/phase sequence.
- Adaptive RL approaches the finite-reference Helstrom limit more closely than prior hand-designed strategies.

Figures intentionally omitted here; the corresponding plots from the article can be inserted later.

Appendix F: frequentist optimization via Fisher information

The library also supports local frequentist design by optimizing the Cramer-Rao bound.

$$K = \mathbb{E}\Big[(\hat{\theta} - \theta)(\hat{\theta} - \theta)^\top\Big] \geq F^{-1}(\theta),$$

with trajectory-level Fisher information

$$F_{ij}(\theta) = \mathbb{E}_y\left[\frac{\partial \log p(y \mid x, \theta)}{\partial \theta_i}\frac{\partial \log p(y \mid x, \theta)}{\partial \theta_j}\right].$$

Contracting with $G \geq 0$ gives the scalar loss

$$L(\lambda) = \mathrm{tr}\Big(GF^{-1}\Big).$$

- The gradient requires differentiating the Fisher matrix through the policy.
- Importance sampling can be used if rare trajectories dominate the Fisher information.

Appendix G: lower bounds and how to benchmark learned policies

Appendix G builds Bayesian Cramer-Rao style lower bounds tailored to the examples.

- For measurement-limited protocols,

$$\Delta^2\hat{\theta} \geq \frac{1}{\sup_h \mathbb{E}_\theta[I(\theta \mid \tau_h)] + I(\pi)} \geq \frac{1}{M\,\mathbb{E}_\theta[\sup_\tau I(\theta \mid \tau)] + I(\pi)}.$$

- For time-limited protocols, the relevant object is the *normalized* Fisher information $I(\theta \mid \tau)/\tau$.
- In NV magnetometry, binary outcomes also imply bit-counting limits that can dominate when the Fisher-information bound is loose.

These bounds define the gray "impossible region" used in the paper's benchmark plots.

Appendix H: backward recursion and the control-theory viewpoint

Appendix H reframes the learned policy as an approximation to a dynamic-programming solution.

- At the last step, the optimal control x^*_{M-1} minimizes the expected terminal loss conditioned on the current PF ensemble.
- Recursing backward defines a family of optimal decision rules $r_t(p_t)$ acting on the particle-filter state.
- The optimization is **non-myopic**: a control at time t is chosen for its effect on the *final* loss, not only the next measurement.

Interpretation

The neural network is not a black box replacing physics; it is a function approximator for the intractable backward-recursion solution on belief states.

Appendix I: backpropagation structure and the stop-gradient trick

The terminal loss can be written as a composition of Bayes-update maps acting on the initial PF ensemble. The derivative expands into repeated products of two kinds of terms:

- direct dependence of the Bayes update on the control selected by the agent,
- indirect dependence through earlier PF states fed back into later policy evaluations.

Appendix I shows that:

- these recursive terms generate expensive higher-order gradient paths,
- inserting a stop-gradient on the PF input,

$$x_{t+1} = F_\lambda\big(\mathrm{sg}[P(\theta \mid x_t, y_t); R_t; t]\big),$$

  removes a large fraction of backward complexity,
- yet the resulting optimization remains non-myopic because each local update still targets final precision.

Take-home messages

1. **Belief-state control:** quantum metrology can be formulated as sequential control on a Bayesian belief state.
2. **Differentiable inference:** particle filtering can be embedded into model-aware RL through soft resampling, reparametrization, and score-function corrections.
3. **Versatility:** the same framework supports adaptive/non-adaptive and Bayesian/frequentist design.
4. **Interpretability:** Appendices H and I reveal the learned policy as an approximation to backward recursion with tractable backpropagation.

Possible closing slide additions: benchmark plots, platform-specific diagrams, and an implementation workflow from `qsensoropt`.